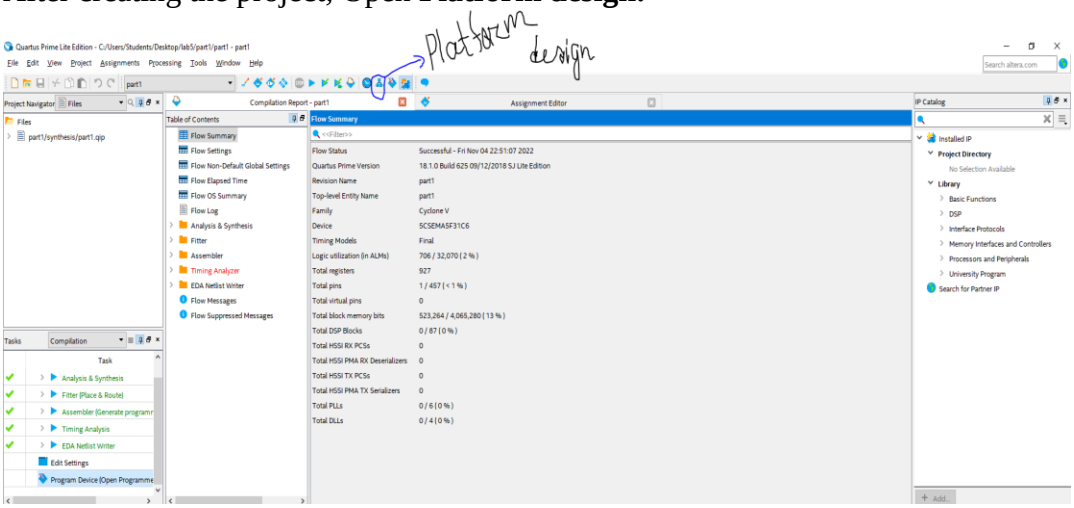
LAB 5 REPORT

Objective: The main objective of lab5 is to implement NIOS II and print helloworld program into NIOS II processor.

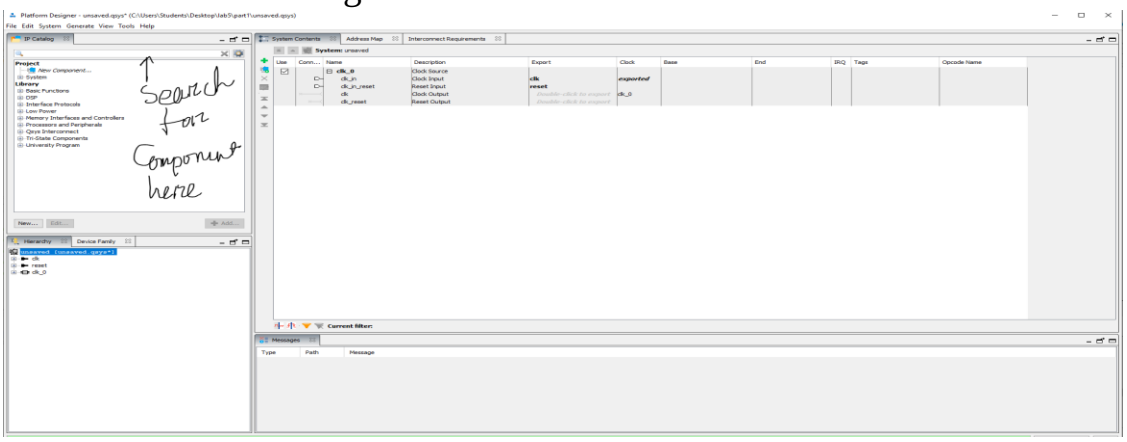
Key Note: I use intel Quartus prime 18.1 lite for this lab 5. Although I tried to solve this with 20.1, but intel introduced WSL Linux subsystem after version 18. WSL is highly dependency and little buggy and not a stable one till date which I feel. Although I designed NIOS with 20.1, but when I build program in NIOS II then it causes several issues.

Part 1: Part1 is to implement NIOS II. Here I will provide the screenshot and the verbal text in a hierarchy manner.

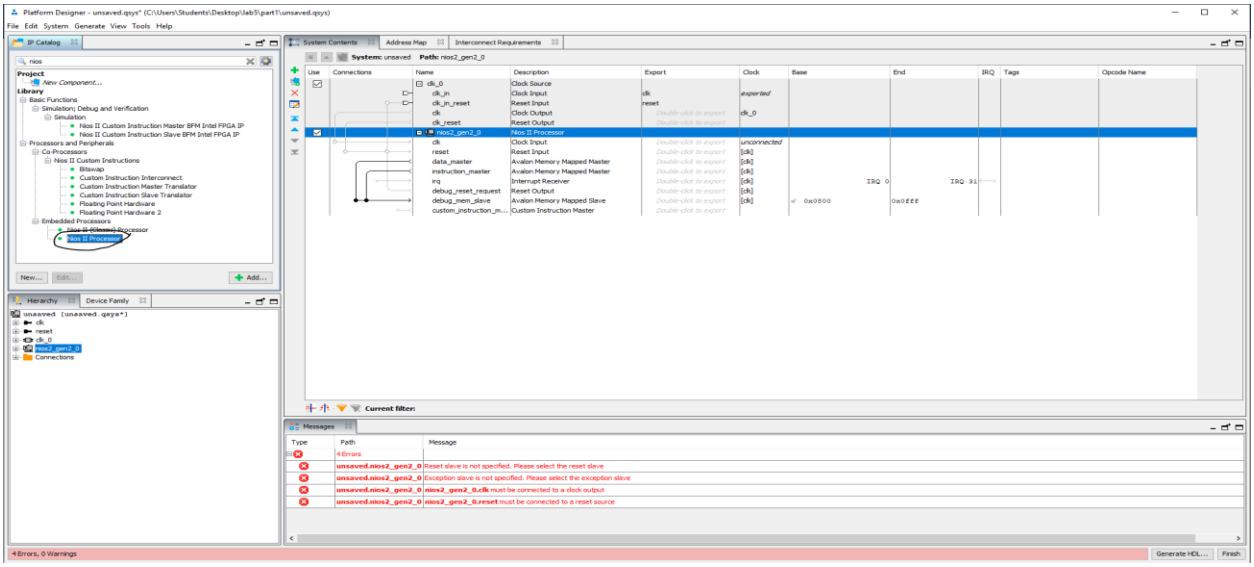
- First, I create a project Wizard what we did in our previous labs.
- After creating the project, Open **Platform design.**

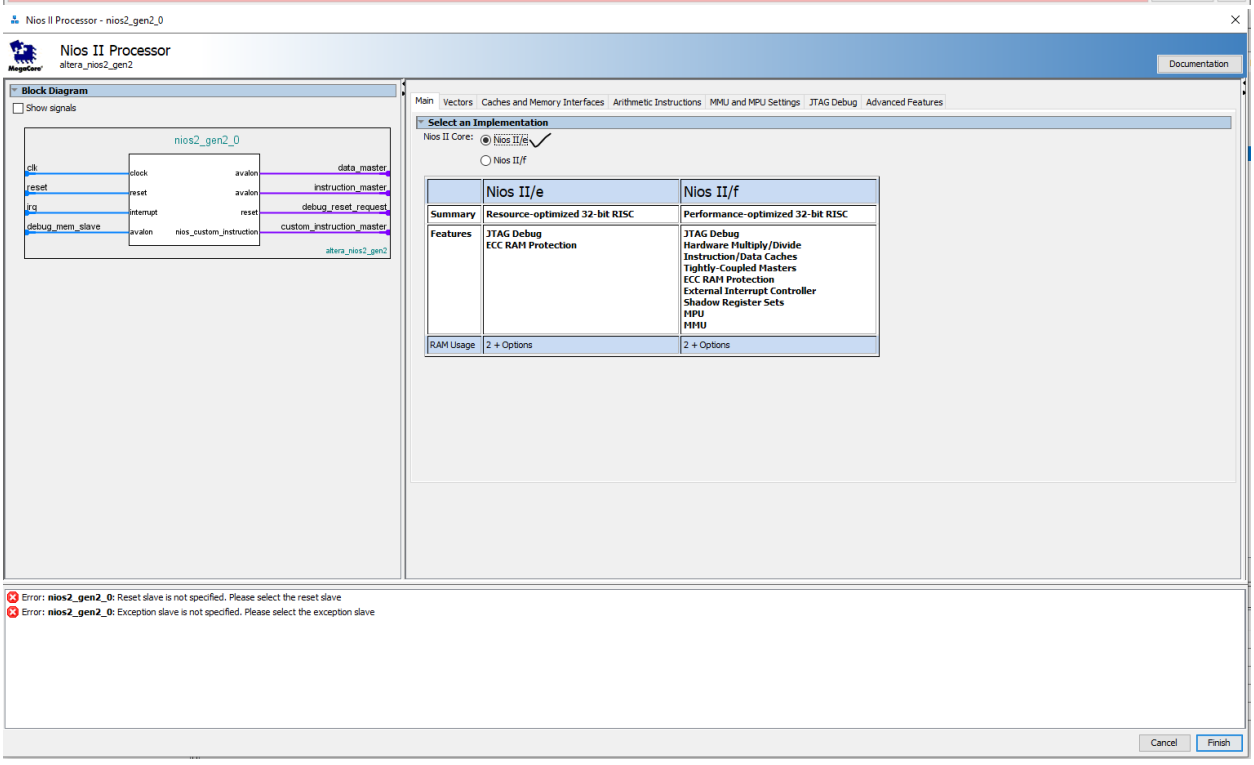


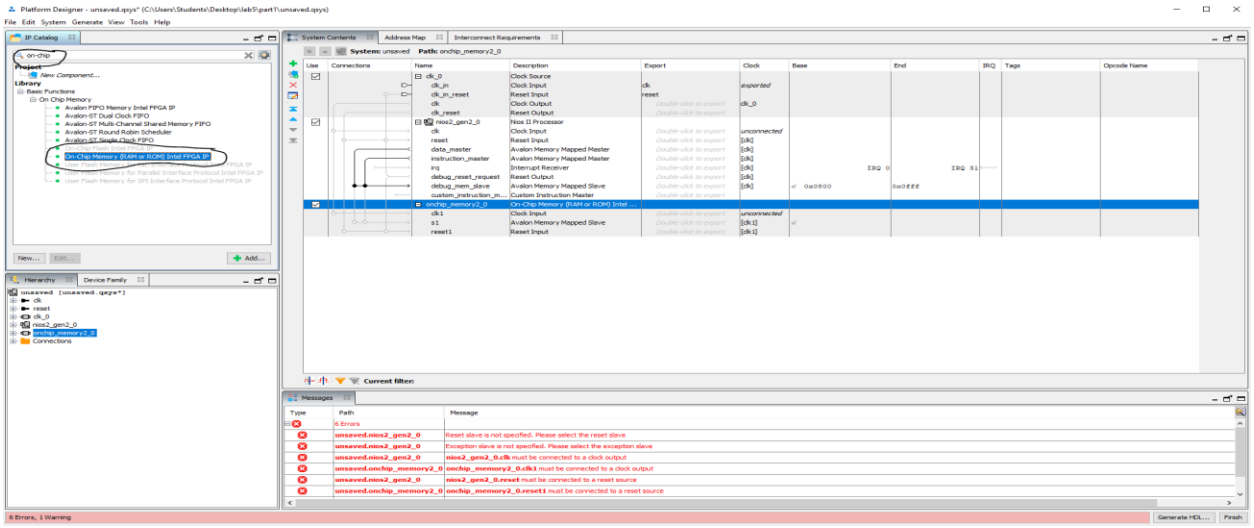
- The platform design will be look like below one and all the component will be found in the marking area.

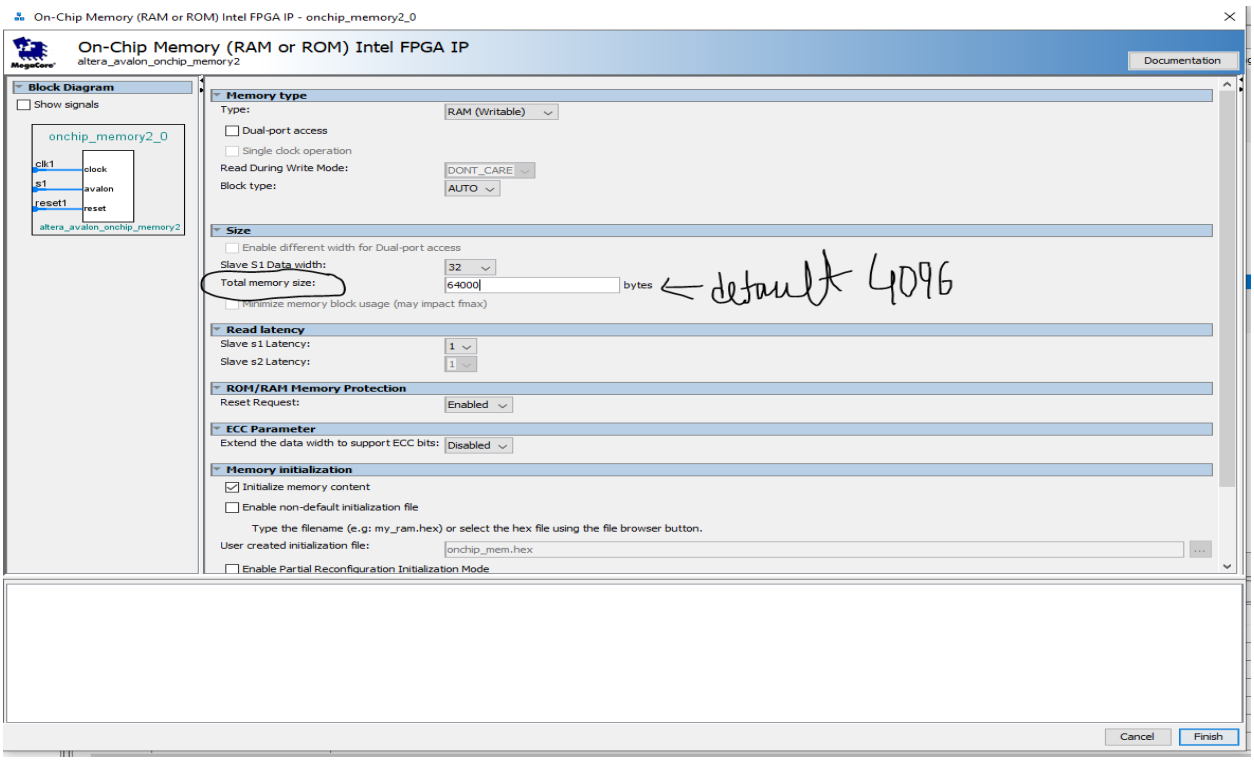


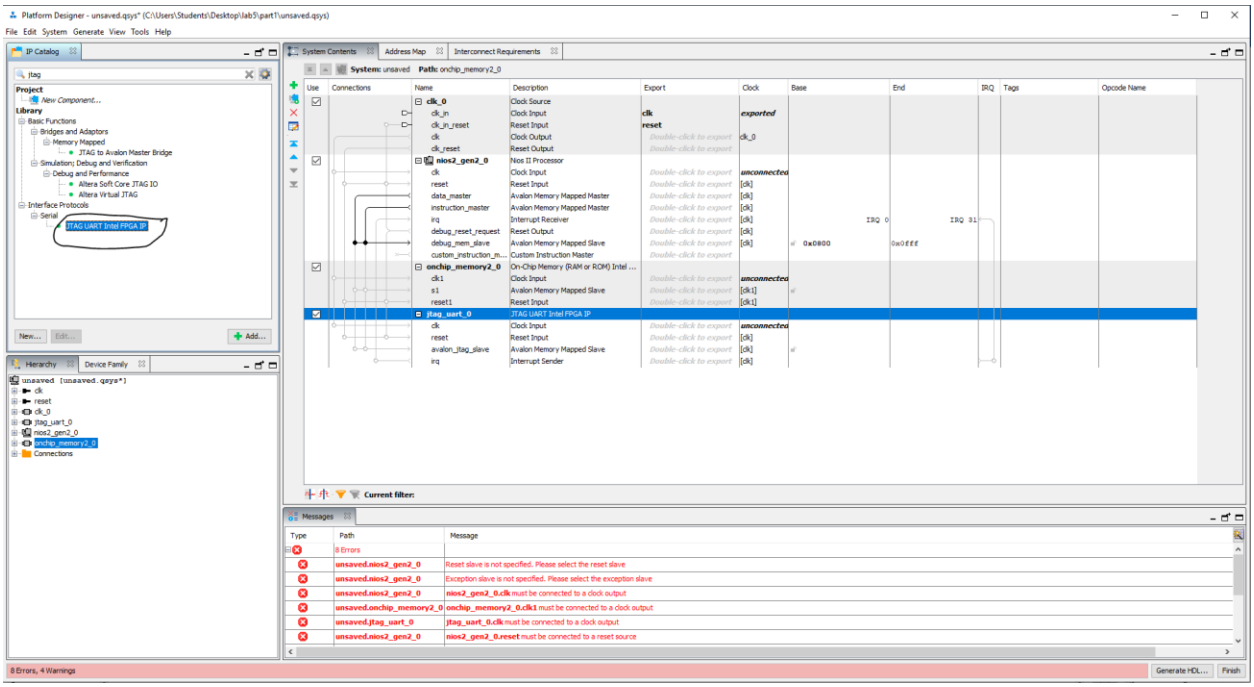
- After that I add NIOS II processor (choose NIOS II/e because it is worked with free version), on chip Memory RAM/ROM, and JTAG UART Intel FPGA IP from the component. In on chip memory, I change the memory to 64000 bytes as with 4096 bytes I got error of elf file not generated. The screenshot of the adding those components is below:

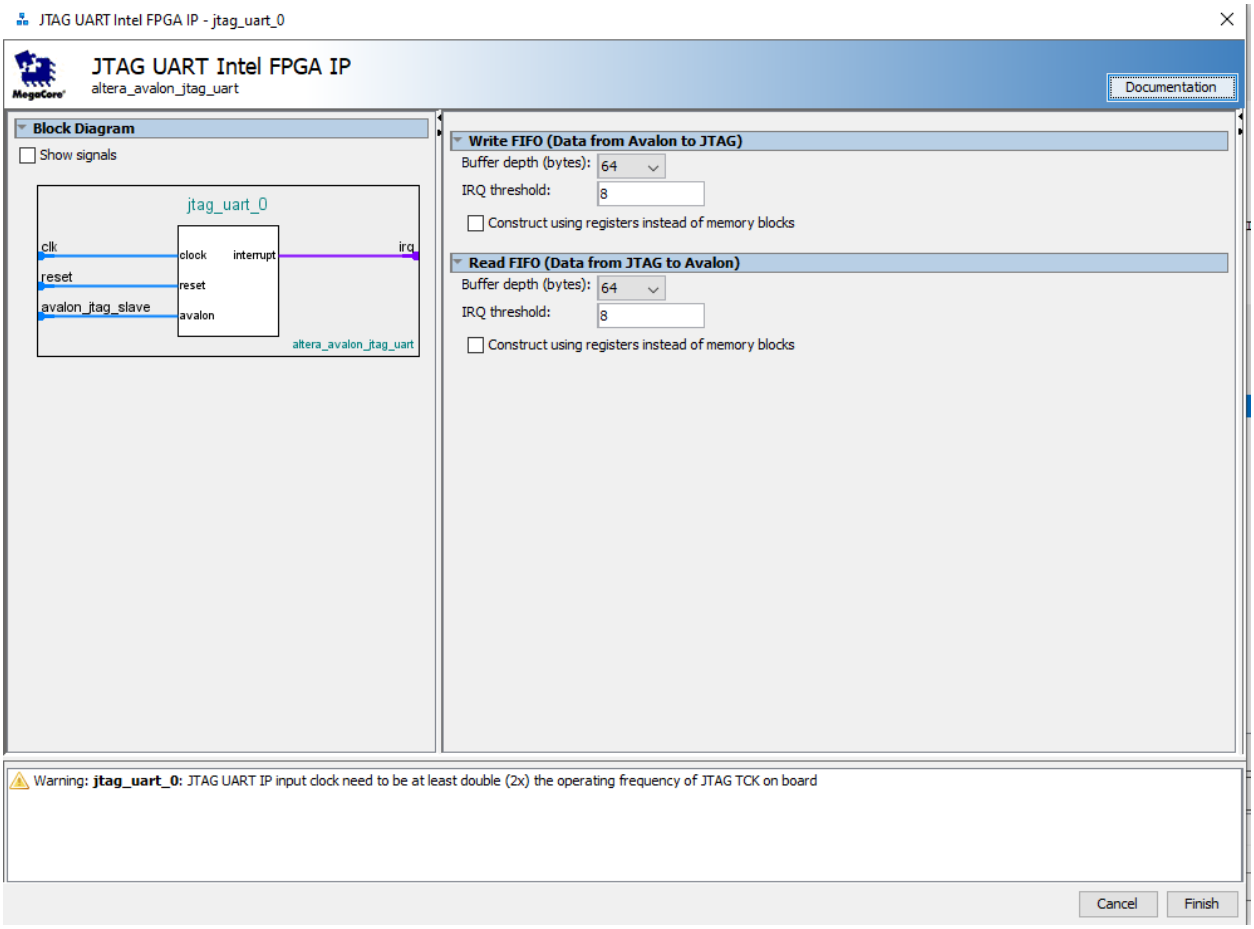




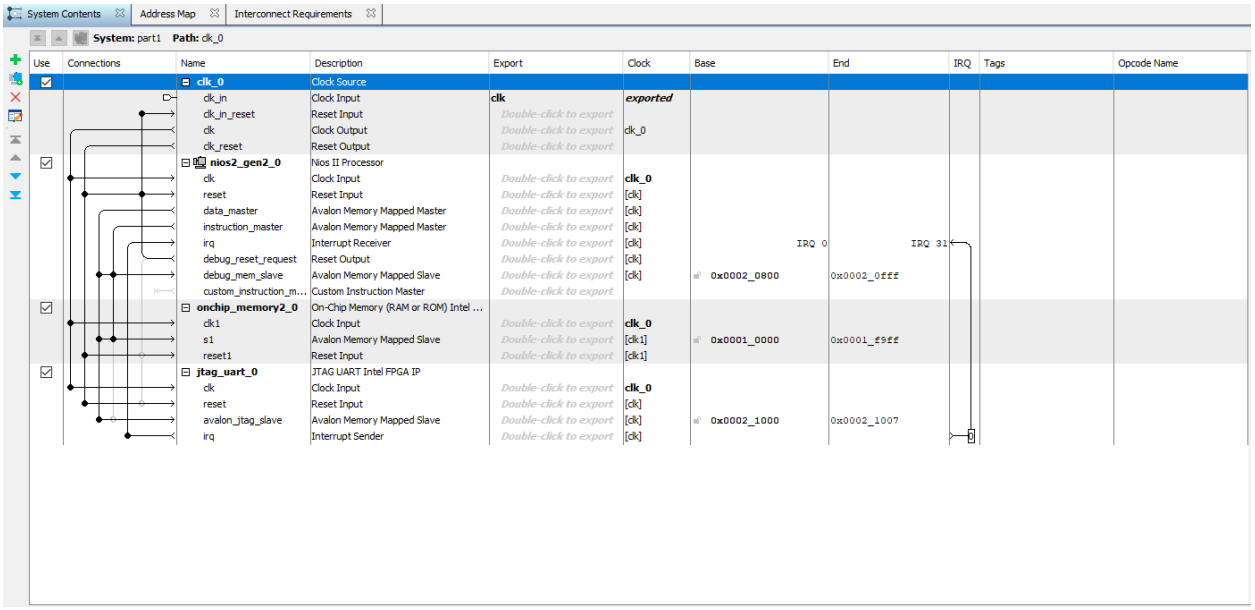

Platform Designer - unsaved.qsys
File Edit System Generate View Tools Help
IP Catalog
System Contents
Address Map
Interconnect Requirements
Hierarchy
Device Family
Messages
Current filter:


On-Chip Memory (RAM or ROM) Intel FPGA IP - onchip_memory2_0
On-Chip Memory (RAM or ROM) Intel FPGA IP
altera_avalon_onchip_memory2
Documentation
Block Diagram
Show signals
onchip_memory2_0
clk1
s1
reset1
clock
avalon
reset
altera_avalon_onchip_memory2
Memory type
Type:
RAM (Writable)
Dual-port access
Single clock operation
Read During Write Mode:
DONT_CARE
Block type:
AUTO
Size
Enable different width for Dual-port access
Slave S1 Data width:
32
Total memory size:
64000
bytes
default 4096
Minimize memory block usage (may impact fmax)
Read latency
Slave s1 Latency:
1
Slave s2 Latency:
1
ROM/RAM Memory Protection
Reset Request:
Enabled
ECC Parameter
Extend the data width to support ECC bits:
Disabled
Memory initialization
Initialize memory content
Enable non-default initialization file
Type the filename (e.g: my_ram.hex) or select the hex file using the file browser button.
User created initialization file:
onchip_mem.hex
Enable Partial Reconfiguration Initialization Mode
Cancel
Finish

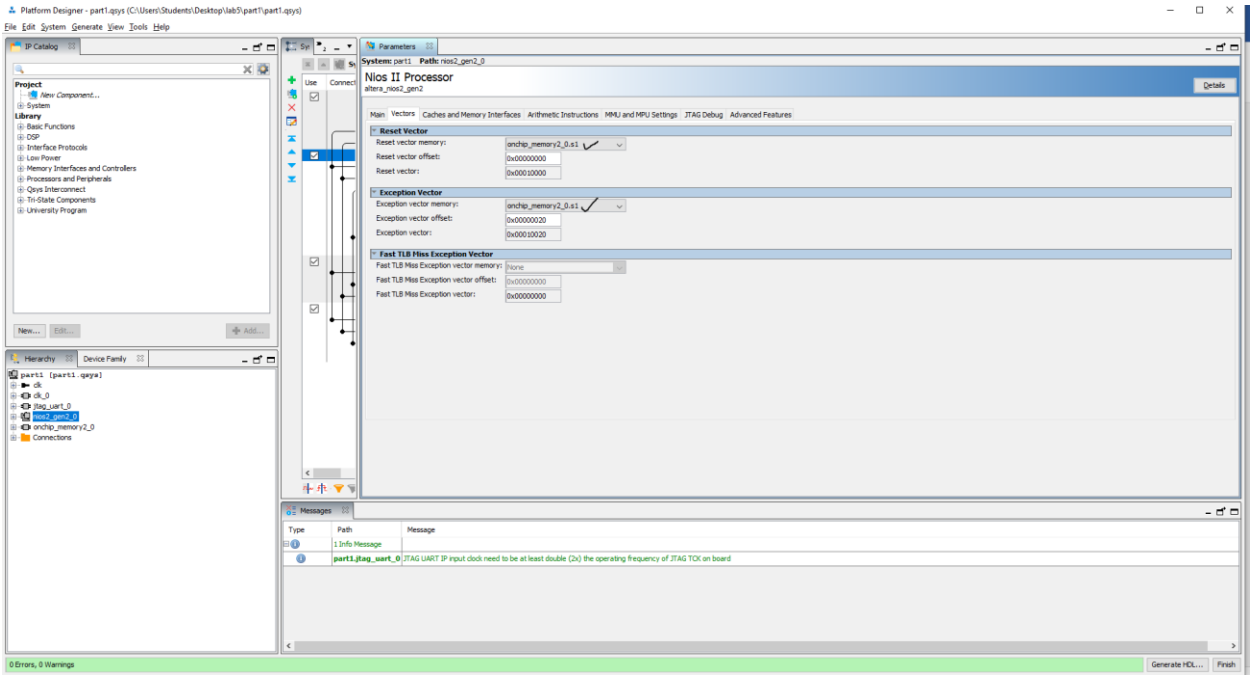

Platform Designer - unsaved.qsys* (C:\Users\Students\Desktop\lab5\part1\unsaved.qsys)
File Edit System Generate View Tools Help
IP Catalog
System Contents
Address Map
Interconnect Requirements
JTAG UART Intel FPGA IP
Hierarchy
Device Family
Messages
Type
Path
Message
8 Errors, 4 Warnings
Generate HDL...
Finish


JTAG UART Intel FPGA IP - jtag_uart_0
JTAG UART Intel FPGA IP
altera_avalon_jtag_uart
Documentation
Block Diagram
Show signals
jtag_uart_0
clk
reset
avalon_jtag_slave
clock
reset
avalon
interrupt
irq
altera_avalon_jtag_uart
Write FIFO (Data from Avalon to JTAG)
Buffer depth (bytes): 64
IRQ threshold: 8
Construct using registers instead of memory blocks
Read FIFO (Data from JTAG to Avalon)
Buffer depth (bytes): 64
IRQ threshold: 8
Construct using registers instead of memory blocks
Warning: jtag_uart_0: JTAG UART IP input clock need to be at least double (2x) the operating frequency of JTAG TCK on board
Cancel
Finish

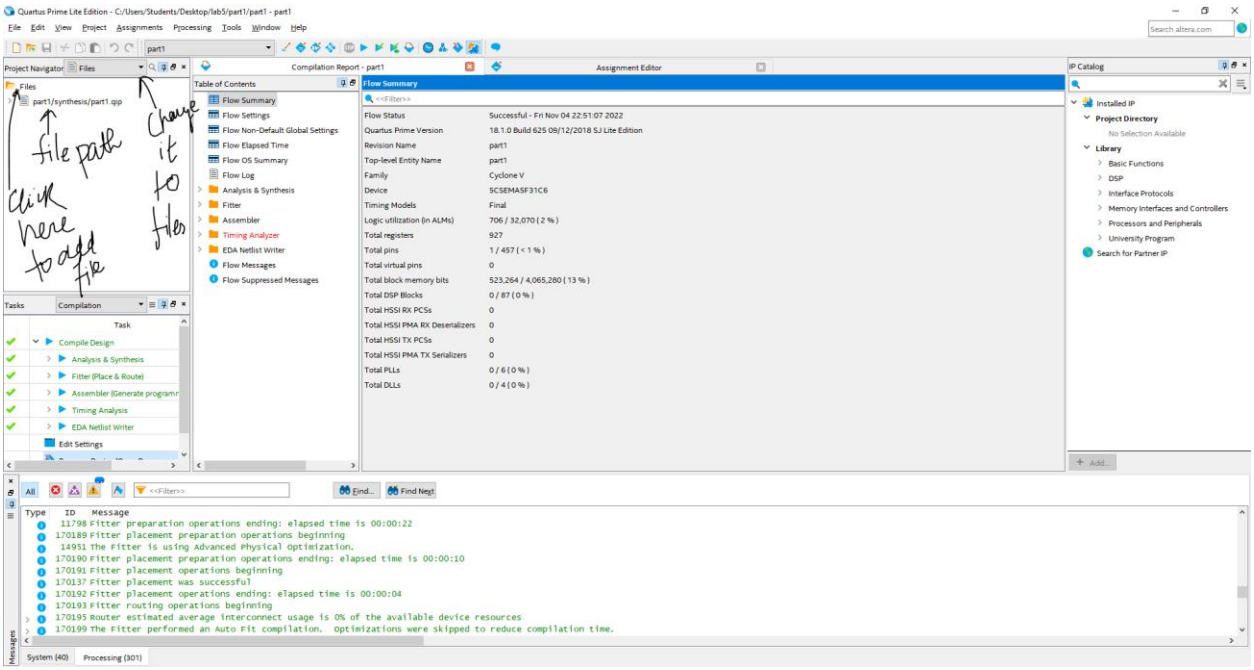
- After adding all the components, I need to wire those components. I need to wire all the clocks, reset button along with instruction master. The complete wire is in below:



- Then I select NIOS II processor and set the vector memory to on chip RAM.



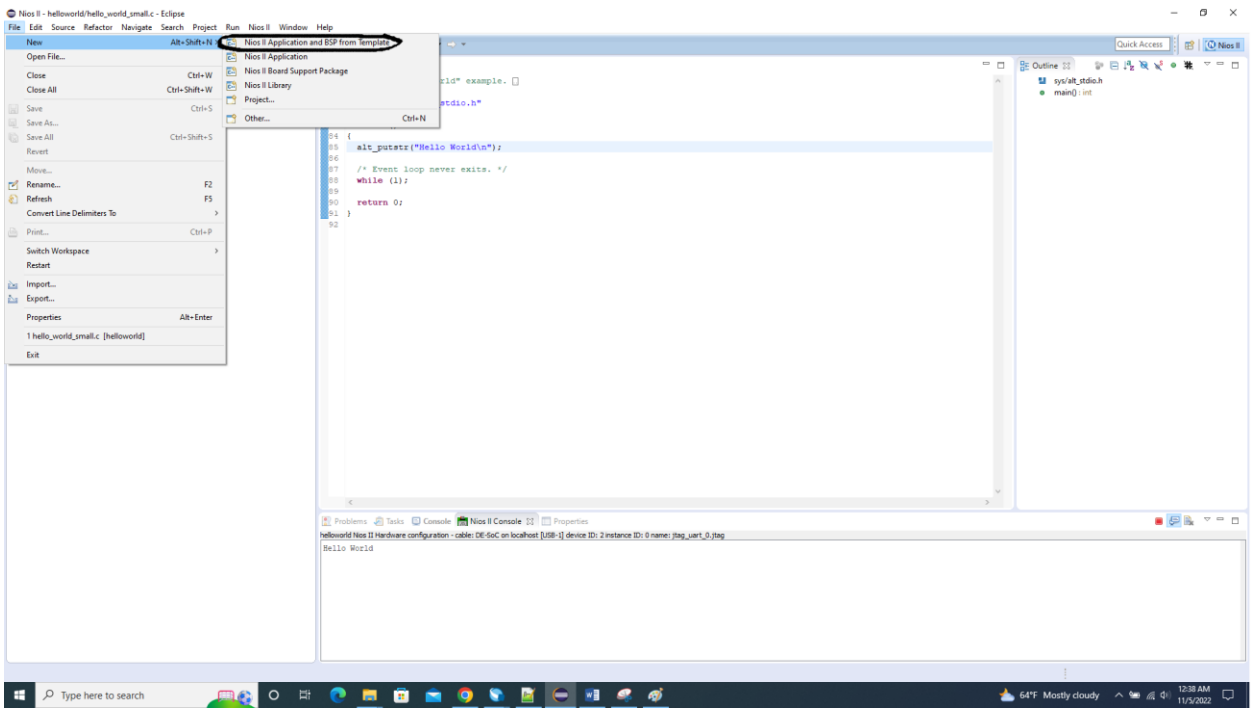
- Then click **system**-> **Assign base addresses**.
- After that click **Generate** which will generate the VHDL and synthesis files.
- Now Close the platform designer and get back to Quartus window.
- In the Quartus we need to first add the **.qip** file clicking the side menu.



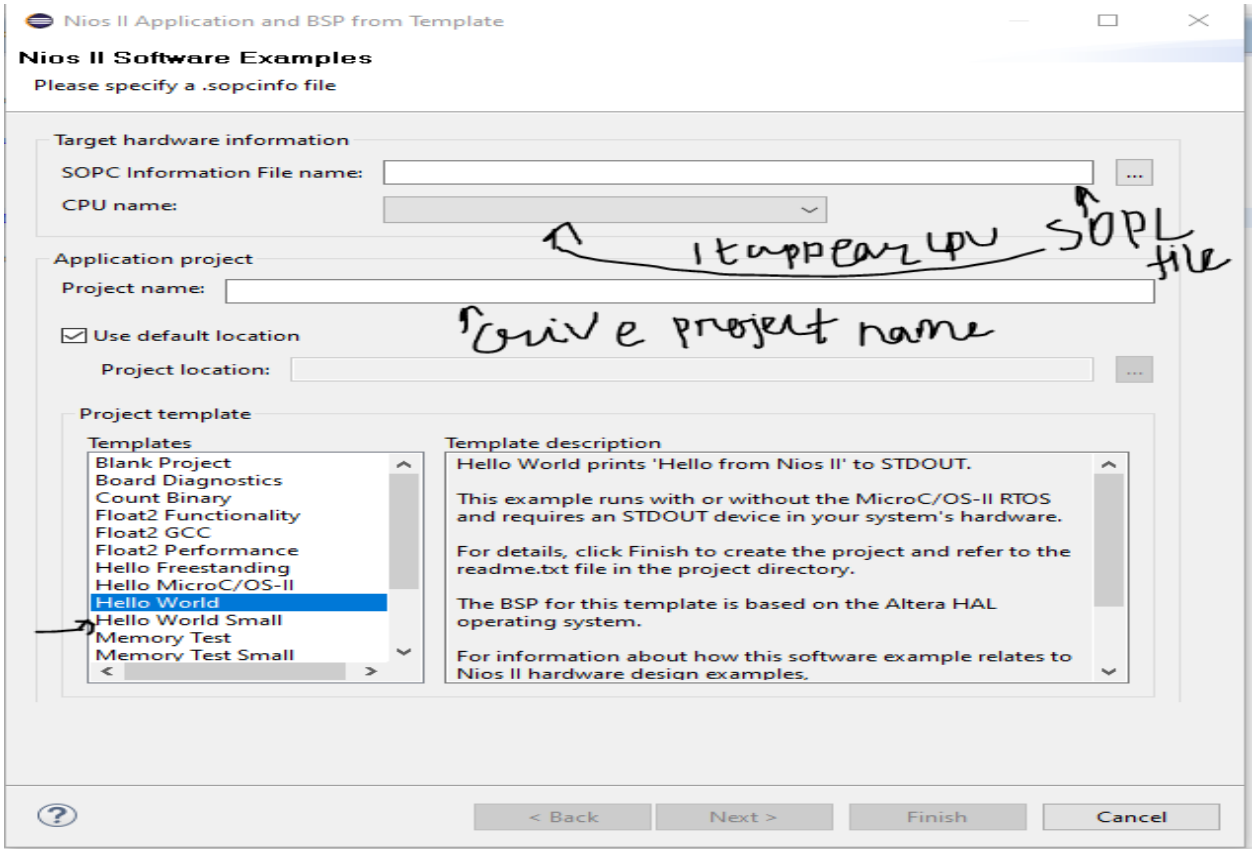
- Compile the design and after successfully compile the design just following the previous method with programmer to implement in DE1-soc board.

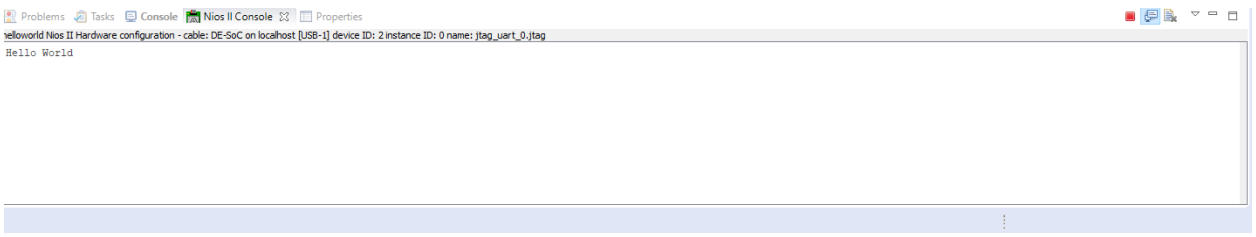
Part 2: In the part 2, I implement hello world in the implemented NIOS II.

- I open the eclipse from the menu in Quartus **tools**-> **NIOS II Software build tools for Eclipse .** It will open the eclipse mars.
- Add new project in eclipse clicking **NIOS II application and BSP from template**



- Add the SOPC files which will be in the Quartus folder with an extension .**sopcinfo**. After that I add the project name and project application folder. **choose Hello world small template .**
- Click finish after adding all the fields will create two files. one is a bsp files which contains all the hardware drives and another is software file which contains program files.
- **Then build the project and run as -> NIOS II hardware. It will show hello world in console.**





Note: All the files are inside the part1 folder. Software files are in part1/software.